

Tervezési Minták (Design Patterns)

Mi az a tervezési minta?

A tervezési minták tipikus megoldások a szoftvertervezés során gyakran felmerülő problémákra. Olyanok, mint az előre elkészített tervrajzok (blueprints), amelyeket testreszabhatsz, hogy megoldj egy visszatérő tervezési problémát a kódodban.

Nem lehet egy mintát csak úgy „lemásolni” és beilleszteni a programba, mint egy kész függvényt vagy könyvtárat. A minta nem egy konkrét kódrészlet, hanem egy **általános koncepció** egy adott probléma megoldására. A minta részleteit követve olyan megoldást implementálhatsz, amely megfelel a saját programod valóságának.

Miben különbözik az algoritmustól?

A mintákat gyakran összekeverik az algoritmusokkal, mivel mindkettő megoldást kínál ismert problémákra.

- **Algoritmus:** Egyértelmű lépések sorozata egy cél eléréséhez (mint egy főzési recept).
- **Tervezési minta:** Magasabb szintű leírása a megoldásnak. A végeredményt és annak tulajdonságait látod, de a megvalósítás pontos sorrendje és részletei rajtad múlnak (mint egy építészeti tervrajz). Ugyanaz a minta két különböző programban teljesen eltérő kódot eredményezhet.

Miből áll egy minta leírása?

A legtöbb minta formális leírással rendelkezik, hogy különféle kontextusokban is alkalmazható legyen:

- **Cél (Intent):** Röviden leírja a problémát és a megoldást.
- **Motiváció (Motivation):** Elmagyarázza a problémát és a minta által kínált megoldást.
- **Osztályszerkezet (Structure):** Bemutatja a minta részeit és azok kapcsolatait.
- **Kódpélda:** Segít megérteni a minta mögötti gondolatot (pl. Java, C++, Python nyelven).

1. Adapter (Adapter)

Más néven: Wrapper (Csomagoló)

Cél

Az Adapter egy szerkezeti (structural) tervezési minta, amely lehetővé teszi, hogy **inkompatibilis interfésszel rendelkező objektumok együttműködjenek**.

Probléma

Képzeld el, hogy egy tőzsdei figyelő alkalmazást készítesz. Az alkalmazás XML formátumban tölt le adatokat, majd diagramokat jelenít meg. Később integrálni szeretnél egy külső, intelligens elemző könyvtárat, de van egy bökkenő: az elemző könyvtár csak JSON formátumot fogad el.

- Nem használhatod a könyvtárat "ahogy van", mert a formátumok nem kompatibilisek.

- Nem írhatod át a külső könyvtárat (gyakran nincs is hozzáférése a forráskódhoz, vagy más kódok függenek tőle).

Megoldás

Létrehozatsz egy **Adaptert**. Ez egy speciális objektum, amely átalakítja az egyik objektum interfészét úgy, hogy egy másik objektum megértse azt.

- Az adapter "becsomagolja" az egyik objektumot, elrejtve a konverzió bonyolultságát.
- A becsomagolt objektum nem is tud az adapter létezéséről.
- Például: Egy adapter átalakíthatja az XML adatokat JSON struktúrává a háttérben, majd továbbítja a kérést az elemző objektumnak.

Való életbeli analógia

Amikor az USA-ból Európába utazol, a laptopod csatlakozója nem illik a konnektorba. A probléma megoldása egy hálózati adapter, amely rendelkezik amerikai típusú aljzattal és európai típusú dugóval.

Működése és Szerkezete

Két fő megvalósítási módja van:

1. **Objektum adapter (Kompozíció):** Az adapter implementálja az egyik objektum interfészét és becsomagolja (tartalmazza) a másikat. Minden népszerű nyelven működik.
2. **Osztály adapter (Öröklődés):** Az adapter egyszerre öröklődik mindkét objektumtól. Csak többszörös öröklődést támogató nyelveknél (pl. C++) lehetséges.

Előnyök és Hátrányok

- **(+) Single Responsibility Principle:** A konverziós kód leválasztható az üzleti logikáról.
- **(+) Open/Closed Principle:** Új adattípusokat vezethetsz be a meglévő kód törése nélkül.
- **(-) Bonyolultság:** A kód komplexitása nő, mivel új osztályokat és interfészeket kell bevezetni.

2. Prototípus (Prototype)

Más néven: Clone (Klón)

Cél

A Prototípus egy létrehozási (creational) tervezési minta, amely lehetővé teszi **objektumok másolását anélkül, hogy a kódod függne azok osztályaitól.**

Probléma

Van egy objektumod, és pontos másolatot szeretnél készíteni róla. A hagyományos módszer (új objektum létrehozása, majd minden mező átmásolása) nem mindig működik:

- Egyes mezők privátak lehetnek, így kívülről nem láthatók.

- Hogy másolatot készíts, ismerned kell az objektum konkrét osztályát, ami függőséget teremt a kódodban.
- Néha csak az interfészt ismered, a konkrét osztályt nem.

Megoldás

A minta a másolási folyamatot magukra a másolandó objektumokra delegálja. Deklarál egy közös interfészt (általában egy clone metódussal) minden objektum számára, amely támogatja a klónozást.

- A clone metódus létrehoz egy új objektumot az aktuális osztályból, és átviszi a régi objektum összes mezőjének értékét az újba.
- Ez lehetővé teszi a privát mezők másolását is, mivel az objektum önmagát másolja.
- Az ilyen objektumot **prototípusnak** nevezzük.

Való életbeli analógia

A biológiai sejtosztódás (mitózis). Az osztódás után két azonos sejt jön létre. Az eredeti sejt prototípusként működik, és aktív szerepet vállal a másolat létrehozásában.

Alkalmazhatóság

- Amikor a kódodnak nem szabad függnie a másolandó objektumok konkrét osztályaitól.
- Amikor csökkenteni szeretnéd az alosztályok számát, amelyek csak az inicializálás módjában különböznek.

Előnyök és Hátrányok

- **(+) Függetlenség:** Klónozhatsz objektumokat anélkül, hogy a konkrét osztályokhoz kötnéd magad.
- **(+) Kevesebb ismétlés:** Megszabadulhatsz az ismétlődő inicializálási kódoktól előre gyártott prototípusok klónozásával.
- **(+) Öröklődés alternatívája:** Kezelheted a komplex objektumok konfigurációs beállításait öröklődés nélkül.
- **(-) Körkörös hivatkozások:** A körkörös hivatkozásokkal rendelkező komplex objektumok klónozása trükkös lehet.

3. Felelősséglánc (Chain of Responsibility)

Más néven: CoR, Chain of Command (Parancslánc)

Cél

A Felelősséglánc egy viselkedési (behavioral) tervezési minta, amely lehetővé teszi, hogy **kéréseket passzolj tovább egy kezelőkből (handlers) álló láncon**. Kérés érkezésekor minden kezelő eldönti, hogy feldolgozza-e a kérést, vagy továbbadja a lánc következő elemének.

Probléma

Képzeld el, hogy egy online rendelési rendszeren dolgozol. A hozzáférést korlátozni kell: csak hitelesített felhasználók rendelhetnek, és az adminoknak teljes hozzáférésük van.

- Az ellenőrzéseket (autentikáció, validáció, gyorsítótárazás) szekvenciálisan kell végrehajtani.
- Ahogy új funkciókat adsz hozzá (pl. brute force elleni védelem), a kód egyre bonyolultabbá és átláthatatlanabbá válik.
- Ha megpróbálsz újrahasznosítani a kód részeit más komponensekben, kódismétléshez vezet.

Megoldás

A minta azt javasolja, hogy alakítsd át az egyes viselkedéseket (ellenőrzéseket) önálló objektumokká, úgynevezett **kezelőkké** (handlers).

- Ezeket a kezelőket láncba kell kötni. Minden kezelő rendelkezik egy hivatkozással a lánc következő elemére.
- A kérés végigmegy a láncon, amíg minden kezelőnek lehetősége nem volt feldolgozni azt.
- A legjobb rész: egy kezelő dönthet úgy, hogy **nem adja tovább** a kérést, ezzel megállítva a folyamatot (pl. sikertelen autentikáció esetén).

Való életbeli analógia

Technikai támogatás hívása.

1. Automata válaszadó (Robot): Próbál megoldást adni. Ha nem sikerül -> továbbadja.
2. Élő operátor: Próbál segíteni. Ha túl bonyolult -> továbbadja.
3. Mérnök: Megoldja a problémát.

Alkalmazhatóság

- Amikor a programnak különféle kéréseket kell feldolgoznia többféle módon, de a kérések típusa és sorrendje nem ismert előre.
- Amikor fontos, hogy több kezelőt hajts végre egy meghatározott sorrendben.
- Amikor a kezelők készletét és sorrendjét futásidőben kell megváltoztatni.

Előnyök és Hátrányok

- **(+) Vezérelhető sorrend:** Szabályozhatod a kéréskezelés sorrendjét.
- **(+) Single Responsibility Principle:** Szétválaszthatod a műveletet kérő és a műveletet végrehajtó osztályokat.
- **(+) Open/Closed Principle:** Új kezelőket adhatsz a lánchoz a meglévő kód törése nélkül.
- **(-) Kezeletlen kérések:** Előfordulhat, hogy egyes kérések végigmennek a láncon anélkül, hogy bárki kezelné őket.